

When Simple Sorting Algorithms Outperform Advanced Ones: An Experimental Study on Small and Large Datasets

Mohammad Gayada
West University of Timișoara
Email: mohamad.ghayada04@e-uvt.ro

April 21, 2026

Abstract

This paper presents an experimental comparison of multiple sorting algorithms, focusing on the conditions under which simple algorithms outperform more advanced ones. While algorithms such as Quick Sort and Merge Sort have better theoretical complexity, experimental results show that simpler algorithms can be more efficient for small datasets. The study also investigates how performance changes when input size increases.

1 Introduction

Sorting algorithms are fundamental in computer science and are widely used in many applications. Advanced algorithms such as Quick Sort and Merge Sort are often preferred due to their optimal time complexity.

However, this paper explores a different perspective: in certain situations, simpler algorithms such as Insertion Sort can outperform more complex ones. This study focuses on understanding when and why this happens.

2 Research Question

This paper aims to answer the following question:

Under what conditions do simple sorting algorithms outperform more advanced algorithms?

3 Algorithms Overview

The following algorithms were analyzed:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Cocktail Sort
- Gnome Sort
- Shell Sort
- Merge Sort
- Quick Sort
- Heap Sort
- Counting Sort
- Radix Sort
- Bucket Sort

4 Experimental Setup

The experiments were implemented in Java using `System.nanoTime()` to measure execution time.

The following input types were tested:

- Random
- Sorted
- Reverse
- Almost Sorted

- Half Sorted
- Few Unique

Experiments were conducted on arrays of size $n = 10$ and $n = 1000$ in order to compare performance on both small and larger datasets.

Testing multiple input sizes allows a better understanding of algorithm scalability and highlights differences between theoretical and practical performance.

The full source code, experimental results, and the PDF version of this paper are available at:

<https://github.com/mohamadghayada04/sorting-benchmark>

The repository includes all implemented algorithms and allows full reproducibility of the experiments.

5 Results

Two different input sizes were tested.

For small datasets ($n = 10$), simpler algorithms such as Insertion Sort and Selection Sort performed best due to low overhead.

For larger datasets ($n = 1000$), more advanced algorithms such as Quick Sort and Merge Sort showed better scalability.

Table 1: Average Execution Time ($n = 10$)

Algorithm	Avg Time (sec)
Insertion Sort	0.000010
Selection Sort	0.000010
Gnome Sort	0.000011
Bubble Sort	0.000012
Cocktail Sort	0.000011
Shell Sort	0.000016
Quick Sort	0.000016
Heap Sort	0.000019
Counting Sort	0.000015
Radix Sort	0.000021
Bucket Sort	0.000020
Merge Sort	0.000028

6 Analysis

The results show that simpler algorithms can outperform advanced algorithms when the input size is small.

This is mainly due to lower overhead and fewer operations compared to recursive algorithms such as Merge Sort.

Insertion Sort performed best overall, while Gnome Sort showed excellent performance on sorted data.

When the input size increased to $n = 1000$, the advantage of simple algorithms decreased, and more efficient algorithms such as Quick Sort and Merge Sort became more competitive.

This confirms theoretical expectations, where $O(n \log n)$ algorithms outperform $O(n^2)$ algorithms for larger inputs.

This highlights that algorithm selection should consider both input size and data characteristics.

7 Limitations

The experiments were conducted on relatively small and medium datasets. Larger datasets may provide more accurate insights into scalability.

Additionally, performance may vary depending on hardware and system conditions.

8 Conclusion

This study demonstrates that simple sorting algorithms can outperform advanced ones under specific conditions, particularly for small or structured datasets.

However, as input size increases, advanced algorithms become more efficient.

The results emphasize the importance of choosing the right algorithm based on context rather than relying solely on theoretical complexity.